

HelixSkills

HelixSkills

CLI AI एजेंटों के लिए एक शासित, संविधान-समर्थित कौशल प्रणाली।

सारांश

HelixSkills, CLI AI एजेंटों के लिए एक कौशल प्रणाली है जो Helix Constitution को एक उप-मॉड्यूल के रूप में अपनाती है, जिससे हर सार्वभौमिक शासन नियम बिना शर्त लागू होता है। यह इंस्टॉल करने योग्य एजेंट कौशल, MCP टूल सर्वर, Claude कोड प्लगइन्स और पुनः उपयोगी इंजनों को एक पंजीकरण योग्य, प्रलेखित कैटलॉग के पीछे संयोजित करता है।

संक्षिप्त विवरण

HelixSkills, CLI AI एजेंटों के लिए एक कौशल प्रणाली है। यह Helix Constitution को उप-मॉड्यूल के रूप में समाहित करता है ताकि सभी सार्वभौमिक नियम लागू हों, फिर पंजीकरण योग्य कौशल (एक्शन-प्रिफिक्स, मीडिया-वैलिडेटर, मल्टीट्रैक, सत्र-सिंक, कार्य योग्य आइटम जीवनचक्र, और अन्य), दो MCP टूल सर्वर, दो Claude कोड प्लगइन्स और पुनः उपयोगी इंजनों को प्रस्तुत करता है।

विस्तृत विवरण

HelixSkills (रिपॉजिटरी skills, Apache-2.0) CLI AI एजेंटों के लिए एक कौशल प्रणाली है, और यह सामान्य क्रम का एक जानबूझकर उलटाव से शुरू होती है: शासन पहले, क्षमता बाद में। यह Helix Constitution को अपने constitution/ उप-मॉड्यूल के रूप में अपनाती है, जिससे constitution/CLAUDE.md और constitution/Constitution.md के हर सार्वभौमिक नियम बिना शर्त लागू होते हैं — न कि किसी परंपरा के रूप में जिसे एजेंट मानने का विकल्प चुन सके, बल्कि एक नियम-सेट के रूप में जो परियोजना के कोड ट्री में भौतिक रूप से समाहित है। जो एजेंट HelixSkills को अपनाता है, वह संविधान से बाहर नहीं निकल सकता; नियम कोड के साथ ही यात्रा करते हैं।

जहाँ अधिकांश “कौशल फ्रेमवर्क” अमूर्त अवधारणाओं का व्यापार करते हैं, वहीं HelixSkills एक ठोस, पंजीकरण योग्य सूची प्रदान करता है जिसे आप इंगित कर सकते हैं और इंस्टॉल कर सकते हैं। सात संविधान कौशल register.sh के माध्यम से इंस्टॉल होते हैं: एक्शन-प्रिफिक्स-सिस्टम, मीडिया-वैलिडेटर, मल्टीट्रैक, रिपोर्टिंग-वर्केबल-

आइटम्स, शेड्यूल्ड-वर्क-क्यू, सत्र-सिंक, और वर्केबल-आइटम-लाइफसाइकल — एक ऐसा विस्तार जो मध्यम से उन्नत स्तर तक फैला है, जिसमें अनुशासित एक्शन नामकरण से लेकर मीडिया सत्यापन और कार्य की इकाई के पूर्ण जीवनचक्र तक सब कुछ शामिल है। अतिरिक्त प्रारूपण कौशल (एंड्रॉइड अवलोकन, Java/Kotlin भाषा, Linux ओएस) पहले से ही अनुक्रमित और सक्रियण की प्रतीक्षा में हैं। दो MCP टूल सर्वर (मीडिया-वैलिडेटर, शेड्यूल्ड-वर्क) उन कौशलों को Model Context Protocol के माध्यम से एजेंटों तक पहुँचाते हैं, जबकि दो Claude कोड प्लगइन्स (हेलिक्स, शेड्यूल्ड-वर्क) उन्हीं क्षमताओं को सीधे एजेंट रनटाइम में समाहित कर देते हैं — एक ही कौशल सेट, जो एजेंट जिस भी माध्यम से संवाद करते हैं, उसके माध्यम से पहुँचता है।

कैटलॉग के नीचे चार गहराई-1 पुनः उपयोगी इंजन स्थित हैं — कॉन्टिनम (कार्यान्वित), साथ ही सत्र ऑर्केस्ट्रेटर, टोकन ऑप्टिमाइज़र, और क्लिकअप सिंक (डिज़ाइन में) — वह साझा मशीनरी जो कौशलों को एक ही प्लंबिंग को बार-बार बनाने से रोकती है। टोकन ऑप्टिमाइज़र अकेले ही वासिक-डिजिटल पारिस्थितिकी तंत्र के पैकेजों (टून, एम्बेडिंग्स, वेक्टरडीबी, नॉर्मलाइज़, कन्वर्सेशन) और HelixDevelopment के LLMProvider तक एक स्पष्ट निर्भरता ग्राफ़ घोषित करता है, जिससे इसकी क्रॉस-रिपॉजिटरी वायरिंग स्पष्ट और ऑडिट योग्य होती है, न कि अंतर्निहित। इसके चारों ओर अनुशासित प्रलेखन चलता है: एक कौशल कैटलॉग, एक स्वतः-जनित कौशल-ग्राफ़ सूचकांक, प्रति-रिपॉजिटरी विस्तृत पृष्ठ, और एक स्पष्ट “गैप्स एंड रिस्क्स” रजिस्टर जो उन चीज़ों को नामित करता है जो अभी तक पूरी नहीं हुई हैं। पूरी प्रणाली GitHub, GitLab, GitFlic और गिटवर्स पर मिरर की गई है ताकि लचीलापन और क्षेत्रीय पहुँच सुनिश्चित हो सके।

सामग्री

हमने इसे क्यों बनाया

CLI और AI एजेंट्स को ऐसी क्षमताओं की जरूरत होती है जो सुसंगत, नियंत्रित और पुनःप्रयोगी हों—न कि हर बार नियमों को नए सिरे से गढ़ने वाले तदर्थ स्क्रिप्ट्स। HelixSkills को एजेंट्स को एक पैकेज्ड, पंजीकरण योग्य कौशल-सेट देने के लिए बनाया गया है जो एक साझा संविधान से बंधा होता है, ताकि हर एजेंट और परियोजना में व्यवहार सुसंगत और ऑडिट योग्य बना रहे जो इसे अपनाता है।

यह क्यों गेम-चेंजर है

यह एजेंट की क्षमता को अनुशासन के बजाय निर्माण के स्तर पर पोर्टेबल और नियम-अनुपालक बनाता है। हर कौशल एक नियंत्रित, संस्करणित, इंस्टॉल करने योग्य इकाई है जो एक संविधान उप-मॉड्यूल द्वारा समर्थित है—इसलिए जैसे ही कोई एजेंट किसी कौशल को पंजीकृत करता है, वह स्वतः ही मानक नियम-सेट को भी विरासत में पा लेता है, जिसमें विचलन की कोई गुंजाइश नहीं रहती। इससे वह संभव हो पाता है जो पहले व्यावहारिक नहीं था: एक क्षमता को एक एजेंट या परियोजना से दूसरी में ले जाना और यह जानना कि वह उसी शासन-व्यवस्था से बंधी पहुँचती है, जो MCP सर्वर और Claude कोड प्लगइन्स जैसे मानक इंटरफेस के माध्यम से उपलब्ध होती है—न कि हर बार नियमों को नए सिरे से गढ़ने वाले तदर्थ गोंद-स्क्रिप्ट्स के ढेर के रूप में।

क्या है नवीन

- **उप-माँडचूल के रूप में Constitution:** सार्वभौमिक शासन-नियम विरासत में मिलते हैं, नकल नहीं—इन्हें ट्री में माउंट किया जाता है ताकि हर उपभोग करने वाला एजेंट उसी मानक नियम-सेट से बंधा रहे, जिसमें अपडेट एक ही स्रोत से प्रवाहित होते हैं, न कि दर्जनों पुराने कॉपियों से।
- **स्व-पंजीकरण इकाइयों के रूप में कौशल (register.sh):** इन्हें एक स्वतः-जनित कौशल-ग्राफ इंडेक्स में जोड़ा जाता है, ताकि कैटलॉग खोजने योग्य बना रहे और कभी भी इंस्टॉल किए गए कौशलों से असंक्रनाइज़ न हो।
- **बहु-सतही प्रदर्शन:** एक ही कौशल-सेट MCP टूल सर्वर और Claude कोड प्लगइन्स के माध्यम से एजेंट्स तक पहुँचता है—एक बार लिखें, जिस भी रनटाइम का एजेंट उपयोग करें, उससे संवाद करें।
- **पुनःप्रयोगी गहराई-1 इंजन (continuum, token optimizer, session orchestrator, clickup sync)** जो पूरे पारिस्थितिकी तंत्र में साझा किए जाते हैं, प्रत्येक में स्पष्ट, ऑडिट योग्य क्रॉस-रिपॉजिटरी निर्भरता घोषणाएँ होती हैं, न कि छिपे हुए जुड़ाव।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **कई कौशलों और एजेंट्स में एजेंट व्यवहार को सुसंगत और नियम-अनुपालक बनाए रखना**—प्रत्येक कौशल के लिए शासन को फिर से लागू करने से समय के साथ विचलन सुनिश्चित होता है। इसका समाधान Helix Constitution को उप-माँडचूल के रूप में माउंट करके किया गया, ताकि constitution/CLAUDE.md और constitution/Constitution.md में दिए गए नियम बिना शर्त लागू हों और एक ही अपस्ट्रीम से अपडेट होते रहें, न कि कॉपी करके छोड़ दिए जाएँ।
- **बढ़ते कौशल-सेट को इंस्टॉल करने योग्य और खोजने योग्य बनाना**—एक कैटलॉग तब बेकार है जब कोई उसमें मौजूद चीज़ों को ढूँढ़ या इंस्टॉल न कर सके। इसका समाधान प्रति-कौशल register.sh पंजीकरण से किया गया, जो इंस्टॉल के समय हर कौशल को जोड़ता है, साथ ही एक स्वतः-जनित INDEX कौशल-ग्राफ और प्रति-रिपॉजिटरी विस्तृत दस्तावेज़ भी दिए गए, ताकि खोज वास्तविकता के अनुरूप बनी रहे।
- **विभिन्न रनटाइम बोलने वाले एजेंट्स तक पहुँचना**—एक ही क्षमता को हर होस्ट के लिए फिर से नहीं बनाया जाना चाहिए। इसका समाधान एक ही कौशल-सेट को MCP टूल-सर्वर परिभाषाओं (constitution/mcp/ के तहत) और Claude कोड प्लगइन्स (constitution/plugins/ के तहत) के पीछे पैक करके किया गया, ताकि एक ही कार्यान्वयन सभी सतहों पर उपलब्ध हो।

सामग्री

तकनीकी संरचना

- **Shell (प्राथमिक भाषा)** — चुना गया क्योंकि एजेंट के रहने वाले किसी भी स्थान पर इंस्टॉल और पंजीकरण टूलिंग को बिना किसी रनटाइम के पहले बूटस्ट्रेप किए चलना होता है; यह register.sh और install_upstreams को शक्ति प्रदान करता है, जिससे आरंभिक निर्भरता-मुक्त और पोर्टेबल बना रहता है।
- **गिट सबमाँडचूल्स** — चुना गया शासन को बिना नकल के विरासत में लेने के लिए: Helix Constitution को constitution/ में एक सक्रिय संदर्भ के रूप में माउंट किया गया है, ताकि नियमों के अपडेट एक पॉइंटर के माध्यम से प्रसारित हों न कि कॉपी-पेस्ट और भुला दिए जाएँ।

- **Model Context Protocol (MCP)** — एजेंटों के लिए मानक, रनटाइम-अज्ञेयवादी टूल इंटरफ़ेस के रूप में चुना गया; दो MCP सर्वर (media-validator, scheduled-work) constitution/mcp/ के तहत परिभाषित किए गए हैं ताकि कौशल को कॉल करने योग्य टूल के रूप में उजागर किया जा सके।
- **Claude कोड प्लगइन्स** — चुना गया ताकि उन कौशलों को बिना किसी अतिरिक्त गोंद के सीधे एजेंट रनटाइम में डाला जा सके; दो प्लगइन्स (helix, scheduled-work) constitution/plugins/ के तहत भेजे जाते हैं, जो एक अलग होस्ट के लिए MCP सतह का प्रतिबिंब हैं।
- **पुनः प्रयोज्य इंजन (continuum, token_optimizer, session_orchestrator, clickup_sync)** — चुने गए ताकि साझा मशीनरी को व्यक्तिगत कौशलों से अलग करके परियोजनाओं में पुनः उपयोग किया जा सके; उदाहरण के लिए, token_optimizer को vasic-digital पैकेजों (TOON, Embeddings, VectorDB, Normalize, conversation) और HelixDevelopment के LLMProvider से घोषित निर्भरताओं के माध्यम से जोड़ा गया है न कि नकल किए गए कोड से।
- **मल्टी-होस्ट गिट मिररिंग (GitHub, GitLab, GitFlic, GitVerse)** — चुना गया ताकि किसी एक होस्ट की विफलता या क्षेत्रीय प्रतिबंध से पहुंच बाधित न हो; एक ही रिपॉजिटरी को चार अलग-अलग प्लेटफॉर्मों पर सक्रिय रखा गया है ताकि लचीलापन और पहुंच सुनिश्चित हो सके।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। सात संविधान कौशल, दो MCP सर्वर और दो प्लगइन्स भेजे जा चुके हैं; प्रारूपणाधीन कौशल अनुक्रमित हैं और सक्रियण की प्रतीक्षा में हैं, जबकि चार गहराई-1 इंजनों में से तीन (session_orchestrator, token_optimizer, clickup_sync) अभी डिज़ाइन चरण में हैं।
- README में परियोजना को helix_skills के रूप में संदर्भित किया गया है; GitHub पर प्रामाणिक पथ HelixDevelopment/skills है। README में उल्लिखित ट्रैकड-फाइंडिंग्स की संख्या एक स्व-रिपोर्टेड आँकड़ा है।

प्राथमिकता स्तर: Helix-प्राथमिक।